

Усовершенствованное управление потоками

В этой главе

- Пулы потоков.
- Управление зависимостями между задачами пула.
- Перехват работы в пуле потоков.
- Прерывание потоков.

В предыдущих главах все управление потоками сводилось к явному созданию объектов `std::thread` для каждого из них. В нескольких местах было видно, насколько это может быть нежелательно, так как вынуждало управлять продолжительностью жизни объектов потоков, определять число потоков, соответствующее объему решаемой задачи и используемому оборудованию, и т. д. Идеальным был бы сценарий, при котором имела бы возможность разделить код на самые мелкие части, которые можно было бы выполнять одновременно, передать их компилятору и библиотеке и сказать: «Распараллельте это для достижения оптимальной производительности». В главе 10 будет показано, что бывают случаи, позволяющие именно так и сделать: если код, требующий распараллеливания, может быть выражен в виде вызова стандартного библиотечного алгоритма, то в большинстве случаев можно будет запросить у библиотеки распараллеливание.

В нескольких примерах повторялась еще одна тема — использования для решения поставленной задачи нескольких потоков с возможностью их более раннего завершения при определенных условиях. Причиной может быть определение результата, или

ошибка, или даже явный запрос пользователя на прерывание операции. Какой бы она ни была, потокам нужно отправить запрос: «Пожалуйста, остановитесь», чтобы они могли отказаться от полученного задания, прибрать за собой и как можно быстрее завершиться.

В этой главе мы рассмотрим приемы управления потоками и задачами и вначале поговорим об автоматическом управлении потоками и распределении задач между ними.

9.1. Пулы потоков

Сотрудникам многих компаний, обычно работающим в офисе, время от времени нужно выезжать к клиентам или поставщикам либо присутствовать на выставке или конференции. Поездки могут быть обязательными, и в какой-нибудь день в разъездах могут быть сразу несколько сотрудников, но для любого конкретного сотрудника перерывы между командировками могут составлять месяцы или даже годы. Поскольку компании не могут выделить в распоряжение каждого сотрудника служебный автомобиль, они зачастую предоставляют пул машин, доступный всему персоналу. Когда сотруднику нужно уехать по делам, он заказывает на определенное время одну из машин пула, а возвращаясь в офис, предоставляет возможность коллегам взять ее. Если в какой-то день свободных машин не окажется, придется перенести поездку на другую дату.

Пул потоков построен по схожему принципу, за исключением того, что совместно применяются *потоки*, а не машины. В большинстве систем нецелесообразно использовать отдельный поток для каждой задачи, когда можно выполнять ее параллельно с другими, но при этом нужно стремиться задействовать доступную конкурентность там, где это возможно. Именно это и позволяет вам создать пул потоков: задачи, которые могут выполняться одновременно, передаются пулу, который помещает их в очередь ожидающих работ. Затем *рабочие потоки* поочередно берут из очереди по одной задаче, выполняют ее и возвращаются в цикл за следующей задачей.

При создании пула потоков решается несколько основных проблем, касающихся его конструкции, например: сколько потоков использовать, каким будет наиболее эффективный способ распределения задач по потокам и нужно ли ожидать завершения задачи. В этом разделе рассмотрим несколько реализаций пула потоков, начиная с самого простого, в которых дается ответ на поставленные вопросы.

9.1.1. Простейший пул потоков

В самом простом виде пул потоков представляет собой фиксированное число *рабочих потоков*, обычно совпадающее со значением, возвращаемым функцией `std::thread::hardware_concurrency()`. Когда появляется работа, вызывается функция, помещающая ее в очередь ожидающих работ. Каждый рабочий поток берет работу из очереди, выполняет поставленную в ней задачу и обращается к очереди за другой работой. В простейшем случае возможность дожидаться завершения задачи не предусматривается. Если она нужна, то синхронизацией придется заняться самостоятельно. Пример реализации такого пула потоков показан в листинге 9.1.

Листинг 9.1. Простой пул потоков

```

class thread_pool
{
    std::atomic_bool done;
    thread_safe_queue<std::function<void()>> > work_queue; ← ❶
❷ → std::vector<std::thread> threads; ← ❸
    join_threads joiner;
    void worker_thread()
    {
        while(!done) ← ❹
        {
            std::function<void()> task;
            if(work_queue.try_pop(task)) ← ❺
            {
                task(); ← ❻
            }
            else
            {
                std::this_thread::yield(); ← ❼
            }
        }
    }
}

public:
    thread_pool():
        done(false), joiner(threads)
    {
        unsigned const thread_count=std::thread::hardware_concurrency(); ← ❽
        try
        {
            for(unsigned i=0;i<thread_count;++i)
            {
                threads.push_back(
                    std::thread(&thread_pool::worker_thread,this)); ← ❾
            }
        }
        catch(...)
        {
            done=true; ← ❿
            throw;
        }
    }
    ~thread_pool()
    {
        done=true; ← ⓫
    }
    template<typename FunctionType>
    void submit(FunctionType f)
    {
        work_queue.push(std::function<void()>(f)); ← ⓬
    }
};

```

В этой реализации есть вектор рабочих потоков ❷, а для управления очередью работ задействуется одна из потокобезопасных очередей из главы 6 ❶. В данном случае пользователи не могут дожидаться завершения задач и возвращать какие-либо

значения, поэтому для инкапсуляции задач можно воспользоваться объектом `std::function<void()>`. Затем функция `submit()` заключает предоставленную ей функцию или вызываемый объект в экземпляр `std::function<void()>` и помещает его в очередь ⑩.

Потоки формируются в конструкторе: из вызова функции `std::thread::hardware_concurrency()` берется число поддерживаемых оборудованием конкурентных потоков ⑧ и создается именно столько потоков, выполняющих компонентную функцию `worker_thread()` ⑨.

Запуск потока может провалиться из-за выдачи исключения, и на этот случай нужно обеспечить остановку всех уже запущенных потоков и ликвидацию последствий их запуска. Задача решается с помощью блока `try-catch`, который при выдаче исключения устанавливает флаг `done` ⑩. Кроме того, для объединения всех потоков используется экземпляр класса `join_threads` из главы 8 ⑤. Этот же прием работает с деструктором: можно установить флаг `done` ⑪, а экземпляр класса `join_threads` обеспечит завершение работы всех потоков, прежде чем пул будет уничтожен. Следует отметить важность порядка объявления компонентов: и флаг `done`, и очередь `worker_queue` должны быть объявлены до вектора потоков `threads`, который, в свою очередь, должен быть объявлен до `joiner`. Тем самым будет гарантировано удаление компонентов в правильном порядке: к примеру, невозможно безопасно удалить очередь, пока не будут остановлены все потоки.

В самой функции `worker_thread` нет ничего сложного: она находится в цикле, ожидая установки флага `done` ④, извлекая между ожиданиями задачи из очереди ⑤ и выполняя их ⑥. Если задач в очереди нет, функция вызывает `std::this_thread::yield()`, чтобы взять небольшой перерыв ⑦ и дать другому потоку возможность поместить какую-нибудь работу в очередь, пока он через некоторое время не возобновит попытки извлечения задачи.

Даже такого простого пула потоков достаточно будет для многих задач, особенно если они полностью независимы друг от друга, не возвращают никаких значений и не выполняют блокирующих операций. Но немало и таких обстоятельств, при которых простой пул потоков не сможет адекватно соответствовать потребностям, и даже таких, при которых он способен вызвать проблемы, связанные с взаимной блокировкой. Кроме того, в простых случаях, возможно, стоит воспользоваться `std::async`, как в многочисленных примерах, приведенных в главе 8. В данной главе будут рассмотрены более сложные реализации пула потоков, имеющие дополнительные свойства, которые либо полнее соответствуют вашим потребностям, либо сужают круг возможных проблем. Начнем с ожидания завершения переданных пулу задач.

9.1.2. Ожидание завершения задач, переданных пулу потоков

В примерах в главе 8, где потоки порождались явным образом, основной поток после распределения работы между потоками всегда дождался завершения работы только что порожденных потоков, чтобы перед возвращением управления вызывающему коду гарантировать завершение всей задачи. При использовании пула потоков нуж-

по дожидаться завершения не самих рабочих потоков, а переданных пулу потоков задач. Это похоже на ожидание фьючерсов в основанных на применении `std::async` примерах из главы 8. А в простом пуле потоков из листинга 9.1 ожидание придется задавать самостоятельно, задействуя механизмы из главы 4 — условные переменные и фьючерсы. Это усложняет код, и лучше было бы дожидаться завершения задач напрямую.

Пеносредственного ожидания завершения задач можно добиться за счет переноса этих усложнений в пул потоков. Можно заставить функцию `submit()` возвращать описатель задачи с информацией, которой затем можно было бы воспользоваться для ожидания завершения задачи. В этот описатель было бы заключено использование условных переменных или фьючерсов, что упростило бы код, задействующий пул потоков.

Частный случай, требующий дождаться завершения порожденной задачи, возникает, когда основной поток нуждается в результате, вычисленном задачей. Он уже встречался в приводимых в этой книге примерах, например в функции `parallel_accumulate()` из главы 2. В данном случае с помощью фьючерсов можно объединить ожидание с передачей результата. В листинге 9.2 показаны изменения, которые нужно внести в код простого пула потоков, чтобы появилась возможность дождаться завершения задачи, а затем передать возвращаемые значения из задачи ожидающему потоку. Поскольку экземпляры класса `std::packaged_task<>` не допускают копирования и могут только *перемещаться*, класс `std::function<>` больше не может использоваться для элементов очереди, так как `std::function<>` требует, чтобы сохраненные функциональные объекты были копируемыми. Вместо него нужно воспользоваться специализированной функцией-оболочкой, способной обрабатывать типы, предназначенные только для перемещений. Это простой максирующий тип класса с оператором вызова функции. Нужны лишь функции обработки, не получающие параметров и возвращающие тип `void`, следовательно, в реализации это будет простой виртуальный вызов.

Листинг 9.2. Пул потоков, позволяющий дождаться завершения задач

```
class function_wrapper
{
    struct impl_base {
        virtual void call()=0;
        virtual ~impl_base() {}
    };
    std::unique_ptr<impl_base> impl;
    template<typename F>
    struct impl_type: impl_base
    {
        F f;
        impl_type(F&& f_): f(std::move(f_)) {}
        void call() { f(); }
    };
public:
    template<typename F>
    function_wrapper(F&& f):
        impl(new impl_type<F>(std::move(f)))
```

```

{}
void operator>() { impl->call(); }
function_wrapper() = default;
function_wrapper(function_wrapper&& other):
    impl(std::move(other.impl))
{}
function_wrapper& operator=(function_wrapper&& other)
{
    impl=std::move(other.impl);
    return *this;
}
function_wrapper(const function_wrapper&)=delete;
function_wrapper(function_wrapper&)=delete;
function_wrapper& operator=(const function_wrapper&)=delete;
};
class thread_pool
{
    thread_safe_queue<function_wrapper> work_queue;
    void worker_thread()
    {
        while(!done)
        {
            function_wrapper task;
            if(work_queue.try_pop(task))
            {
                task();
            }
            else
            {
                std::this_thread::yield();
            }
        }
    }
public:
    template<typename FunctionType>
    std::future<typename std::result_of<FunctionType()>::type> ← ❶
        submit(FunctionType f)
    {
        typedef typename std::result_of<FunctionType()>::type
            result_type;
        std::packaged_task<result_type()> task(std::move(f)); ← ❷
        std::future<result_type> res(task.get_future()); ← ❸
        work_queue.push(std::move(task)); ← ❹
        return res; ← ❺
    }
    // все остальное не изменилось
};

```

Использование
function_wrapper
вместо std::function

Сначала отметим, что измененная функция `submit()` ❶ возвращает объект `std::future<>` для хранения возвращаемого задачей значения и предоставления вызывающему коду возможности ждать завершения задачи. Для этого требуется знать возвращаемый тип предоставленной функции `f`, и здесь пригодится `std::result_of<>`: типом результата, возвращаемого вызовом экземпляра типа `FunctionType`

(например, `f`) без аргументов, будет `std::result_of<FunctionType()>::type`. То же самое выражение `std::result_of<>` используется для `result_type` typedef ② внутри функции.

Затем функция `f` помещается в оболочку `std::packaged_task<result_type()>` ⑤, поскольку `f` является функцией или вызываемым объектом, не получающим параметры и возвращающим, как выяснилось, экземпляр типа `result_type`. Теперь можно получить фьючерс из `std::packaged_task<>` ④ перед помещением задачи в очередь ⑤ и возвращением фьючерса ⑥. Следует отметить, что при помещении задачи в очередь необходимо задействовать функцию `std::move()`, поскольку `std::packaged_task<>` не допускает копирования. Теперь, чтобы можно было справиться с ситуацией, в очереди хранятся объекты `function_wrapper`, а не объекты `std::function<void()>`.

Этот пул позволяет дожидаться завершения задач и получать от них результаты. В листинге 9.3 показано, какой становится функция `parallel_accumulate` с этим пулом потоков.

Листинг 9.3. Функция `parallel_accumulate`, использующая пул потоков, позволяющий дожидаться завершения задач

```
template<typename Iterator,typename T>
T parallel_accumulate(Iterator first,Iterator last,T init)
{
    unsigned long const length=std::distance(first,last);
    if(!length)
        return init;
    unsigned long const block_size=25;
    unsigned long const num_blocks=(length+block_size-1)/block_size; ←①
    std::vector<std::future<T> > futures(num_blocks-1);
    thread_pool pool;

    Iterator block_start=first;
    for(unsigned long i=0;i<(num_blocks-1);++i)
    {
        Iterator block_end=block_start;
        std::advance(block_end,block_size);
        futures[i]=pool.submit(=[]{
            accumulate_block<Iterator,T>()(block_start,block_end);
②→    });
        block_start=block_end;
    }
    T last_result=accumulate_block<Iterator,T>()(block_start,last);
    T result=init;
    for(unsigned long i=0;i<(num_blocks-1);++i)
    {
        result+=futures[i].get();
    }
    result += last_result;
    return result;
}
```

Если сравнить этот код с кодом листинга 8.4, можно заметить две особенности. Работа проводится в понятиях количества используемых блоков (`num_blocks`) ①, а не количества потоков. Чтобы максимально воспользоваться возможностью

масштабирования пула потоков, нужно разбить работу на мельчайшие блоки, с которыми имеет смысл работать в режиме конкурентности. Если количество потоков в пуле невелико, каждый из них будет предназначен для обработки множества блоков, но по мере роста числа потоков, поддерживаемых оборудованием, будет расти и число блоков, обрабатываемых в параллельном режиме.

При выборе мельчайших блоков, с которыми имеет смысл работать в конкурентном режиме, нужно проявлять осмотрительность. Передача задачи пулу потоков, запуск ее в рабочем потоке и передача возвращаемого значения посредством `std::future<>` не обойдется без издержек, и при мелких задачах они могут обеспечить всю эту затею. Если выбрать слишком маленький размер задачи, то с пулом потоков код может работать медленнее, чем с одним потоком.

При правильном размере блока беспокоиться за упаковку задач, получение фьючерсов или сохранение объектов `std::thread`, чтобы впоследствии можно было объединить потоки, уже не придется, обо всем этом позаботится пул потоков. Останется лишь вызвать функцию `submit()`, передав ей задачу 2.

Пул потоков обеспечит также безопасность выдачи исключений. Любое исключение, выданное задачей, будет распространено посредством фьючерса, возвращаемого из вызова функции `submit()`, и, если выход из функции не обходится без исключения, деструктор пула потоков закроет любые еще не оконченные задачи и дождется завершения потоков, относящихся к пулу.

Для простых случаев, когда задачи не зависят друг от друга, эта конструкция вполне работоспособна. Но для ситуаций, при которых задачи зависят от других задач, также передаваемых пулу потоков, она непригодна.

9.1.3. Задачи, ожидающие завершения других задач

Алгоритм Quicksort многократно приводился в этой книге в качестве примера. В принципе, в нем нет ничего сложного: сортируемые данные разбиваются на элементы, составляющие последовательности до опорного элемента и после него. Эти два набора элементов сортируются в рекурсивном режиме, после чего собираются в полностью отсортированный набор. При распараллеливании алгоритма необходимо, чтобы рекурсивные вызовы пользовались доступными возможностями конкурентности.

При первом представлении этого примера в главе 4 для запуска одного из рекурсивных вызовов на каждом этапе применялась функция `std::async`, что позволяло библиотеке выбирать между запуском этой функции в отношении нового потока и сортировкой в синхронном режиме при вызове соответствующей функции `get()`. Эта реализация работала неплохо, поскольку каждая задача либо запускалась в собственном потоке, либо вызывалась по мере необходимости.

При новом обращении к этой реализации в главе 8 была показана альтернативная структура, использующая фиксированное число потоков, соответствующее возможностям аппаратной конкурентности. В этом случае задействовался стек ожидающих сортировки блоков. Каждый поток, разбив на части сортируемые им данные, добавлял к стеку новый блок, относящийся к одному набору данных, а затем непосредственно сортировал другой набор данных. На этой стадии простое

ожидание завершения сортировки другого блока могло превратиться во взаимную блокировку, поскольку пришлось бы воспользоваться одним из ограниченного числа ожидающих потоков. Все это легко могло вылиться в ситуацию, при которой все потоки ожидали бы завершения сортировки блоков и ни один из них не занимался бы сортировкой. Проблему удалось решить, заставив потоки извлекать блоки из стека и выполнять их сортировку, пока конкретный блок, который они ожидали, был еще не отсортирован.

Если в пример из главы 4 вместо `std::async` подставить простой пул потоков вроде того, который до сих пор рассматривался в этой главе, возникнет точно такая же проблема. Теперь она связана с ограниченным числом потоков, и может оказаться, что все они будут ждать завершения задачи, которая не была запланирована из-за отсутствия свободных потоков. Поэтому следует воспользоваться решением, похожим на одно из использованных в главе 8: обработать остающиеся блоки, пока длится ожидание завершения сортировки пужного блока. Если пул потоков задействуется для управления списком задач и их связью с потоками — в конце концов, в этом и состоит смысл применения пула потоков, — то для этого нам не хватает доступа к списку задач. Пужно изменить пул потоков, чтобы все выполнялось автоматически.

Проще всего добавить к `thread_pool` новую функцию, чтобы запускать задачу из очереди и управлять циклом самостоятельно. Так мы и сделаем. Усовершенствованная реализация пула потоков могла бы включать логику в функцию ожидания или дополнительные функции ожидания, справляющиеся с данным случаем, возможно, за счет расстановки приоритетов задачам, завершения которых ожидаем. В листинге 9.4 показана новая функция `run_pending_task()`, а использующий ее измененный алгоритм Quicksort приведен в листинге 9.5.

Листинг 9.4. Реализация функции `run_pending_task()`

```
void thread_pool::run_pending_task()
{
    function_wrapper task;
    if(work_queue.try_pop(task))
    {
        task();
    }
    else
    {
        std::this_thread::yield();
    }
}
```

Эта реализация `run_pending_task()` полностью перенесена из основного цикла функции `worker_thread()`, которую теперь можно будет изменить, чтобы она вызывала извлеченную функцию `run_pending_task()`. Эта функция пытается извлечь задачу из очереди и выполнить ее, если она там есть, в противном случае уступает управление, позволяя операционной системе перепланировать поток. Реализация Quicksort, показанная в листинге 9.5, намного проще соответствующей версии из листинга 8.1, поскольку вся логика управления потоками была перенесена в пул потоков.

Листинг 9.5. Реализация Quicksort, основанная на использовании пула потоков

```

template<typename T>
struct sorter ← ❶
{
    thread_pool pool; ← ❷

    std::list<T> do_sort(std::list<T>& chunk_data)
    {
        if(chunk_data.empty())
        {
            return chunk_data;
        }
        std::list<T> result;
        result.splice(result.begin(), chunk_data, chunk_data.begin());
        T const& partition_val = *result.begin();
        typename std::list<T>::iterator divide_point =
            std::partition(chunk_data.begin(), chunk_data.end(),
                [&](T const& val){ return val < partition_val; });
        std::list<T> new_lower_chunk;
        new_lower_chunk.splice(new_lower_chunk.end(),
            chunk_data, chunk_data.begin(),
            divide_point);
        ❸ → std::future<std::list<T> > new_lower =
            pool.submit(std::bind(&sorter::do_sort, this,
                std::move(new_lower_chunk)));
        std::list<T> new_higher(chunk_data);
        result.splice(result.end(), new_higher);
        while(new_lower.wait_for(std::chrono::seconds(0)) ==
            std::future_status::timeout)
        {
            pool.run_pending_task(); ← ❹
        }
        result.splice(result.begin(), new_lower.get());
        return result;
    }
};

template<typename T>
std::list<T> parallel_quick_sort(std::list<T> input)
{
    if(input.empty())
    {
        return input;
    }
    sorter<T> s;
    return s.do_sort(input);
}

```

Как и в листинге 8.1, реальная работа делегирована компонентной функции `do_sort()`, принадлежащей шаблону класса `sorter` ❶, хотя в данном случае он нужен только для заключения в него экземпляра `thread_pool` ❷.

Теперь управление потоками и задачами свелось к отправке задачи в пул ❸ и запуску ожидающих выполнения задач в период ожидания ❹. Это намного проще, чем в листинге 8.1, где приходилось явным образом управлять потоками и стеком подлежащих сортировке блоков. При отправке задачи пулу используется функция

`std::bind()`, чтобы привязать указатель `this` к `do_sort()` и предоставить блок для сортировки. В данном случае, как только блок будет передан, в отношении `new_lower_chunk` вызывается функция `std::move()`, чтобы гарантировать, что данные перемещаются, а не копируются.

Теперь важная проблема, вызывавшая взаимную блокировку, связанную с задачами, ожидающими результатов выполнения других задач, решена, однако этот пул потоков еще далек от идеала. Начнем с того, что при каждом вызове `submit()` и `run_pending_task()` происходит обращение к одной и той же очереди. В главе 8 было показано, как наличие одного набора данных, изменяемого несколькими потоками, может ухудшить производительность, поэтому с данной проблемой нужно что-то делать.

9.1.4. Предотвращение конкуренции при обращении к очереди работ

При каждом вызове потоком функции `submit()` в отношении конкретного экземпляра пула потоков приходится помещать новый элемент в единственную совместно используемую очередь работ. Кроме того, рабочие потоки постоянно извлекают элементы из очереди, чтобы запускать задачи. Следовательно, по мере роста числа процессоров растет и конкуренция при обращении к очереди. Это может нанести производительности реальный урон: даже если задействуется очередь, свободная от блокировок, и отсутствует явное ожидание, существенным пожирателем времени может стать перебрасывание данных в кэш-памяти процессоров.

Один из способов, позволяющий избежать перебрасывания, заключается в применении отдельной очереди работ для каждого потока. Тогда каждый поток помещает новые элементы в собственную очередь и берет работу из глобальной очереди работ, только если в его отдельной очереди работы нет. В листинге 9.6 показана реализация, использующая переменную `thread_local` для того, чтобы гарантировать, что наряду с глобальной очередью у каждого потока имеется собственная очередь работ.

Листинг 9.6. Пул потоков с локальными относительно потоков очередями работ

```
class thread_pool
{
    threadsafe_queue<function_wrapper> pool_work_queue;
    typedef std::queue<function_wrapper> local_queue_type;  ← ❶
    static thread_local std::unique_ptr<local_queue_type>
❷ → local_work_queue;
    void worker_thread()
    {
        local_work_queue.reset(new local_queue_type);  ← ❸

        while(!done)
        {
            run_pending_task();
        }
    }
public:
```

```

template<typename FunctionType>
std::future<typename std::result_of<FunctionType()>::type>
submit(FunctionType f)
{
    typedef typename std::result_of<FunctionType()>::type result_type;
    std::packaged_task<result_type> task(f);
    std::future<result_type> res(task.get_future());
    4 → if(local_work_queue)
    {
        local_work_queue->push(std::move(task));
    }
    else
    {
        pool_work_queue.push(std::move(task)); ← 5
    }
    return res;
}
void run_pending_task()
{
    function_wrapper task;
    if(local_work_queue && !local_work_queue->empty()) ← 6
    {
        task=std::move(local_work_queue->front());
        local_work_queue->pop();
        task();
    }
    else if(pool_work_queue.try_pop(task)) ← 7
    {
        task();
    }
    else
    {
        std::this_thread::yield();
    }
}
// все остальное не изменилось
};

```

Для хранения локальной относительно потока очереди работ используется указатель `std::unique_ptr<>` 2, потому что не хочется, чтобы такую же очередь имели другие потоки, не входящие в ваш пул потоков. Этот указатель инициализируется в функции `worker_thread()` до запуска цикла обработки 5. Деструктор `std::unique_ptr<>` гарантирует уничтожение очереди работ, как только произойдет выход из потока.

Затем функция `submit()` проверяет, есть ли у текущего потока очередь работ 4. Если это так, значит, он входит в пул потоков и задачу можно помещать в локальную очередь. В противном случае задача, как и раньше, должна быть помещена в очередь пула 5.

Похожая проверка есть и в функции `run_pending_task()` 6, но на этот раз нужно также выяснить, есть ли в локальной очереди какие-либо элементы. Если есть, можно извлечь крайний элемент из очереди и обработать его. Заметьте: поскольку к локальной очереди обращается всего один поток, она может быть простым объектом `std::queue<>` 1. Если задач в локальной очереди нет, то, как и раньше, выполняется попытка обращения к очереди пула 2.

Со снижением конкуренции код справляется, но при неравномерном распределении работ запросто может случиться так, что у одного потока скопится большой объем работы, а другим будет нечем заняться. Например, в примере Quicksort в очередь пула попадает только самый первый блок, поскольку оставшиеся блоки окажутся в локальной очереди того рабочего потока, который обрабатывал самый первый блок. Это лишает использование пула потоков всякого смысла.

К счастью, у данной проблемы есть решение: нужно позволить потокам *перехватывать* работу из очередей друг друга, если ее нет и в их очереди, и в глобальной очереди.

9.1.5. Перехват работы

Чтобы позволить безработному потоку брать работу у потока с полной очередью, последняя должна быть доступна перехватывающему потоку из функции `run_pending_tasks()`. Для этого требуется, чтобы каждый поток зарегистрировал свою очередь в пуле потоков или получил очередь от пула. Кроме того, нужно обеспечить данным в очереди работ удобную синхронизацию и защиту, чтобы ваши инварианты были надежно защищены.

Можно создать очередь, свободную от блокировок, которая позволяет владеющему ею потоку помещать элементы в один конец и извлекать их оттуда, чтобы другие потоки могли перехватывать элементы с другого конца. Реализация такой очереди не относится к темам данной книги. Чтобы продемонстрировать саму идею, обратимся для защиты данных очереди к использованию мьютекса. Мы надеемся, что перехват работы останется редким событием, поэтому конкуренция за овладение мьютексом будет невелика, следовательно, и издержки у данной простой очереди будут сведены к минимуму. Простая реализация на основе блокировок показана в листинге 9.7.

Листинг 9.7. Очередь на основе блокировок, предназначенная для перехвата работы

```
class work_stealing_queue
{
private:
    typedef function_wrapper data_type;
    std::deque<data_type> the_queue;      ← ❶
    mutable std::mutex the_mutex;
public:
    work_stealing_queue()
    {}
    work_stealing_queue(const work_stealing_queue& other)=delete;
    work_stealing_queue& operator=(
        const work_stealing_queue& other)=delete;
    void push(data_type data)            ← ❷
    {
        std::lock_guard<std::mutex> lock(the_mutex);
        the_queue.push_front(std::move(data));
    }
    bool empty() const
    {
        std::lock_guard<std::mutex> lock(the_mutex);
        return the_queue.empty();
    }
    bool try_pop(data_type& res)         ← ❸
```

```

{
    std::lock_guard<std::mutex> lock(the_mutex);
    if(the_queue.empty())
    {
        return false;
    }
    res=std::move(the_queue.front());
    the_queue.pop_front();
    return true;
}
bool try_steal(data_type& res)  ←❶
{
    std::lock_guard<std::mutex> lock(the_mutex);
    if(the_queue.empty())
    {
        return false;
    }
    res=std::move(the_queue.back());
    the_queue.pop_back();
    return true;
}
};

```

Эта очередь является простой оболочкой вокруг `std::deque<function_wrapper>` ❶, защищающей все обращения блокировкой мьютекса. Функции `push()` ❷ и `try_pop()` ❸ работают в начале очереди, а функция `try_steal()` ❹ — в ее конце.

Следовательно, для владельца эта очередь является стеком, работающим по схеме «последним пришел, первым ушел»: только что помещенная в очередь задача станет первой на выход. Это может помочь поднять производительность за счет удобства копирования, поскольку вероятность того, что данные, относящиеся к этой задаче, все еще будут оставаться в кэше, намного выше, чем вероятность присутствия там данных задачи, помещенной в очередь чуть раньше. Кроме того, все это хорошо согласуется с такими алгоритмами, как Quicksort. В предыдущей реализации при каждом вызове `do_sort()` в стек помещался один элемент, а затем его приходилось ждать. То, что первым обрабатывается последний помещенный в очередь элемент, гарантирует, что блок, необходимый для завершения текущего вызова, будет обработан до тех блоков, которые нужны в других ветвях. Тем самым уменьшится количество активных задач и общая интенсивность использования стека. Чтобы снизить конкуренцию, функция `try_steal()`, в отличие от функции `try_pop()`, берет элементы с противоположного конца очереди. В принципе, чтобы разрешить конкурентный вызов функций `try_pop()` и `try_steal()`, можно было бы воспользоваться приемом, рассмотренным в главах 6 и 7.

Итак, в нашем распоряжении появилась весьма привлекательная очередь работ, допускающая перехваты. Как же ею можно воспользоваться в пуле потоков? Одна из возможных реализаций показана в листинге 9.8.

Листинг 9.8. Пул потоков, использующий перехват работы

```

class thread_pool
{
    typedef function_wrapper task_type;
    std::atomic_bool done;

```



```

threadsafe_queue<task_type> pool_work_queue;
std::vector<std::unique_ptr<work_stealing_queue> > queues; ←❶
std::vector<std::thread> threads;
join_threads joiner;
static thread_local work_stealing_queue* local_work_queue; ←❷
static thread_local unsigned my_index;
void worker_thread(unsigned my_index_)
{
    my_index=my_index_;
    local_work_queue=queues[my_index].get(); ←❸
    while(!done)
    {
        run_pending_task();
    }
}
bool pop_task_from_local_queue(task_type& task)
{
    return local_work_queue && local_work_queue->try_pop(task);
}
bool pop_task_from_pool_queue(task_type& task)
{
    return pool_work_queue.try_pop(task);
}
bool pop_task_from_other_thread_queue(task_type& task) ←❹
{
    for(unsigned i=0;i<queues.size();++i)
    {
        unsigned const index=(my_index+i+1)%queues.size(); ←❺
        if(queues[index]->try_steal(task))
        {
            return true;
        }
    }
    return false;
}
public:
thread_pool():
    done(false),joiner(threads)
{
    unsigned const thread_count=std::thread::hardware_concurrency();
    try
    {
        for(unsigned i=0;i<thread_count;++i)
        {
            queues.push_back(std::unique_ptr<work_stealing_queue>( ←❻
                new work_stealing_queue));
        }
        for(unsigned i=0;i<thread_count;++i)
        {
            threads.push_back(
                std::thread(&thread_pool::worker_thread,this,i));
        }
    }
    catch(...)
    {
        done=true;
    }
}

```

```

        throw;
    }
}
~thread_pool()
{
    done=true;
}
template<typename FunctionType>
std::future<typename std::result_of<FunctionType()>::type> submit(
    FunctionType f)
{
    typedef typename std::result_of<FunctionType()>::type result_type;
    std::packaged_task<result_type()> task(f);
    std::future<result_type> res(task.get_future());
    if(local_work_queue)
    {
        local_work_queue->push(std::move(task));
    }
    else
    {
        pool_work_queue.push(std::move(task));
    }
    return res;
}
void run_pending_task()
{
    task_type task;
    if(pop_task_from_local_queue(task) ||           ← ⑦
       pop_task_from_pool_queue(task) ||           ← ⑧
       pop_task_from_other_thread_queue(task))     ← ⑨
    {
        task();
    }
    else
    {
        std::this_thread::yield();
    }
}
};

```

Это код похож на код листинга 9.6. Первое отличие состоит в том, что у каждого потока есть очередь `work_stealing_queue`, а не простая очередь `std::queue<>` ②. При создании поток не выделяет память под собственную очередь работ — этим занимается конструктор пула ⑥, после чего очередь сохраняется в списке очередей работ для этого пула ①. Затем индекс очереди в списке передается функции потока и используется для получения указателя на очередь ⑤. Это означает, что пул потоков может обращаться к очереди при попытке перехвата задачи для безработного потока. Теперь функция `run_pending_task()` попытается получить задачу из очереди, принадлежащей своему потоку ②, потом из очереди пула ⑧ или из очереди другого потока ⑨.

Функция `pop_task_from_other_thread_queue()` ④ обходит очереди, принадлежащие всем потокам пула, пытаясь перехватить задачу поочередно из каждой. Чтобы избежать попыток перехвата каждым потоком из очереди того потока, который значится первым в списке, все потоки начинают обход с потока, следующего в списке за счет смещения индекса в очереди относительно собственного индекса ⑤.

Теперь у нас имеется работоспособный пул потоков, пригодный для множества вариантов использования. И по-прежнему существует много способов его усовершенствования для любого конкретного применения. Но эту возможность я оставляю читателю в качестве упражнения. Один из неисследованных аспектов — динамическое изменение размера пула потоков, гарантирующее оптимальное использование центрального процессора, даже когда потоки заблокированы в ходе ожидания чего-либо вроде завершения операции ввода-вывода или снятия блокировки мьютекса.

Следующим в списке усовершенствованных приемов управления потоками значится прерывание потоков.

9.2. Прерывание потоков

В ряде ситуаций неплохо было бы иметь возможность просигнализировать долгоиграющему потоку, что пора остановиться. Возможной причиной будет принадлежность рабочего потока к пулу, который уже уничтожен, или же явное прерывание пользователем работы, выполняемой потоком, и множество других обстоятельств. Какой бы ни была причина, суть одна: нужен сигнал от одного потока другому, что он должен остановиться до естественного завершения выполняемой им работы. И сделать это следует таким образом, чтобы потоку было позволено прервать ее элегантно, вместо того чтобы резко прерывать процесс.

В принципе, можно было бы разработать отдельный механизм для каждого подходящего случая, но это явно будет излишним. Общий механизм не только упростит написание кода в дальнейшем, но и позволит создавать код, выполнение которого можно будет прерывать, не переживая за то, где конкретно он будет использоваться. Стандарт C++11 не предоставляет такой механизм (хотя предложение добавить поддержку прерывания в будущий стандарт C++ выдвигается весьма активно¹), но создать его самостоятельно не так уж трудно. Посмотрим, как это можно сделать, и начнем рассматривать вопрос с позиции интерфейса для запуска и прерывания потока, а не с позиции самого прерываемого потока.

9.2.1. Запуск и прерывание другого потока

Для начала рассмотрим внешний интерфейс. Что нам понадобится от потока, допускающего свое прерывание? На самом простом уровне — такой же интерфейс, как у `std::thread`, дополненный функцией `interrupt()`:

```
class interruptible_thread
{
public:
    template<typename FunctionType>
    interruptible_thread(FunctionType f);
    void join();
    void detach();
```

¹ P0660: A Cooperatively Interruptible Joining Thread, Rev 3, Nicolai Josuttis, Herb Sutter, Anthony Williams, <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0660r3.pdf>.

```

    bool joinable() const;
    void interrupt();
};

```

Для управления самим потоком можно в выполняемом в нем коде воспользоваться `std::thread` и задействовать для обработки прерывания какую-нибудь специализированную структуру данных управления. А как это будет выглядеть с позиции самого потока? Все должно сводиться к его возможности сказать: «Я могу быть прерван в этом месте», то есть нужна *точка прерывания*. Чтобы не приходилось передавать дополнительные данные, требуется простая функция `interruption_point()`, которую можно было бы вызывать без каких-либо параметров. Тем самым подразумевается, что специализированная структура данных прерываний должна быть доступна через переменную `thread_local`, значение которой устанавливается при запуске потока, чтобы при вызове потоком функция `interruption_point()` проверяла структуру данных для текущего исполняемого потока. Реализация `interruption_point()` будет рассмотрена чуть позже.

Главной причиной использования флага `thread_local` является невозможность применения для управления потоком только класса `std::thread`: память для него должна быть распределена таким образом, чтобы доступ к флагу имелся как у экземпляра `interruptible_thread`, так и у только что запущенного потока. Для этого, как показано в листинге 9.9, прежде чем передавать предоставляемую функцию классу `std::thread`, чтобы запустить поток в конструкторе, ее можно заключить в оболочку.

Листинг 9.9. Основная реализация `interruptible_thread`

```

class interrupt_flag
{
public:
    void set();
    bool is_set() const;
};

thread_local interrupt_flag this_thread_interrupt_flag; ←❶
class interruptible_thread
{
    std::thread internal_thread;
    interrupt_flag* flag;
public:
    template<typename FunctionType>
    interruptible_thread(FunctionType f)
    {
        std::promise<interrupt_flag*> p; ←❷
        ❶→ internal_thread=std::thread([f,&p]{
            p.set_value(&this_thread_interrupt_flag);
            ❸→ f();
        });
        flag=p.get_future().get(); ←❹
    }
    void interrupt()
    {
        if(flag)
        {

```

```

        flag->set();    ←❷
    }
};

```

Предоставляемая функция `f` заключена в лямбда-функцию ❶, в которой хранятся копия `f` и ссылка на локальный промис `p` ❷. Перед вызовом копии предоставленной функции ❸ лямбда-функция устанавливает для нового потока в качестве значения промиса адрес `this_thread_interrupt_flag`, объявленный с модификатором `thread_local` ❹. Затем вызывающий поток дожидается готовности фьючерса, связанного с промисом, и сохраняет результат в компонентной переменной `flag` ❺. Следует отметить, что факт запуска лямбда-функции в новом потоке и наличие в ней висячей ссылки на локальную переменную `p` не должны вас беспокоить, поскольку конструктор `interruptible_thread` перед возвращением управления ждет, пока на `p` не останется ссылок в новом потоке. Замечу, что в этой реализации не берется в расчет управление объединением с потоком или отсоединением от него. Об очистке переменной `flag` во избежание появления висячего указателя при выходе из потока или отсоединения от него придется позаботиться самостоятельно.

Теперь конструкция функции `interrupt()` представляется относительно простой: если имеется действительный указатель на флаг прерывания, значит, есть поток, который требуется прервать, и флаг можно установить ❹. Что затем делать с этим прерыванием, решает сам прерываемый поток. Далее посмотрим, как это происходит.

9.2.2. Обнаружение того, что поток был прерван

Теперь можно установить флаг прерывания, но толку от этого не будет, пока поток не проверит наличия требования о его прерывании. В самом простом варианте это делается с помощью функции `interruption_point()`: можно вызвать ее в тот момент, когда прерывание потока будет безопасным, и если флаг установлен, она выдаст исключение `thread_interrupted`:

```

void interruption_point()
{
    if(this_thread_interrupt_flag.is_set())
    {
        throw thread_interrupted();
    }
}

```

Этой функцией можно воспользоваться, вызывая ее в коде там, где это удобно:

```

void foo()
{
    while(!done)
    {
        interruption_point();
        process_next_item();
    }
}

```


Решение работоспособное, но не идеальное. Некоторые наиболее подходящие места для прерывания потока находятся там, где он заблокирован в ожидании чего-то, следовательно, не выполняется и не в состоянии вызвать `interruption_point()`. Здесь необходимо средство, позволяющее находиться в состоянии прерываемого ожидания.

9.2.3. Прерывание ожидания на условной переменной

Итак, обозначить прерывания в тщательно подобранных местах кода можно явным вызовом `interruption_point()`, но этот прием не поможет там, где нужно заблокировать поток в ожидании, к примеру, уведомления от условной переменной. Здесь требуется еще одна функция под названием `interruptible_wait()`, которую затем можно будет переопределить под различные варианты ожидания, и нужно выработать способы прерывания ожидания. Как уже упоминалось, один из вариантов — ожидание уведомления от условной переменной. Поэтому с него и начнем: что нужно сделать, чтобы получить возможность прерывать ожидание на условной переменной? Самым простым работоспособным вариантом могут стать уведомление условной переменной об установке флага прерывания и установка точки прерывания сразу же после ожидания. Но чтобы он сработал и гарантированно пробудил нужный поток, придется уведомлять все потоки, находящиеся в режиме ожидания на условной переменной. Но ожидающие потоки в любом случае должны обрабатывать ложные пробуждения, поэтому другие потоки обработают данное явление точно так же, как и ложное пробуждение, — они не смогут отличить одно от другого. В структуре `interrupt_flag` нужно предусмотреть возможность хранения указателя на условную переменную, чтобы ее можно было уведомить в вызове функции `set()`. Одна из возможных реализаций `interruptible_wait()` для условных переменных может выглядеть так, как показано в листинге 9.10.

Листинг 9.10. Нерабочая версия `interruptible_wait` для `std::condition_variable`

```
void interruptible_wait(std::condition_variable& cv,
                      std::unique_lock<std::mutex>& lk)
{
    interruption_point();
    this_thread_interrupt_flag.set_condition_variable(cv); ← ❶
❷ → cv.wait(lk);
    this_thread_interrupt_flag.clear_condition_variable(); ← ❸
    interruption_point();
}
```

Если предположить, что есть функции для установки и удаления связи условной переменной с флагом прерывания, все в этом коде покажется простым и понятным. В нем проверяется наличие прерывания, связанного с условной переменной, имеющей флаг прерывания `interrupt_flag` для текущего потока ❶, выполняется ожидание на условной переменной ❷, устраняется связь с условной переменной ❸ и снова проверяется наличие прерывания. Если поток прерывается в процессе ожидания на условной переменной, то прерывающий поток пошлет сигнал этой переменной и выведет нужный поток из ожидания, позволяя ему проверить наличие прерывания. К сожалению, этот код *не будет работать*: в нем есть две проблемы.

Если озаботиться безопасностью выдачи исключений, то распознать первый из них довольно просто: `std::condition_variable::wait()` может выдать исключение, поэтому выйти из функции можно без удаления связи флага прерывания с условной переменной. Эту проблему легко устранить, если обзавестись структурой, удаляющей связь в своем деструкторе.

Второй, менее очевидный проблемный момент связан с возникновением состояния гонки. Если поток прерывается после начального вызова `interruption_point()`, но перед вызовом `wait()`, то связь условной переменной с флагом прерывания не играет роли, поскольку *поток не находится в режиме ожидания, следовательно, его нельзя пробудить уведомлением от условной переменной*. Нужно обеспечить невозможность уведомить поток в период между последней проверкой на наличие прерывания и вызовом `wait()`. Если не внедряться во внутреннее устройство `std::condition_variable`, остается только один способ справиться с этой задачей: воспользоваться для защиты мьютексом, удерживаемым `lk`, для чего потребуется передать его в вызов `set_condition_variable()`. К сожалению, при этом возникают новые проблемы: ссылка на мьютекс, время жизни которого неизвестно, передается другому потоку (тому, который выполняет прерывание) для прерывания нужного потока (в вызове `interrupt()`), и при этом неизвестно, заблокирован он на момент вызова или нет. Тем самым создаются предпосылки возникновения взаимной блокировки и возможность обращения к мьютексу после того, как он был уничтожен, то есть этот вариант никуда не годится. Отсутствие надежного способа прерывания ожидания на условной переменной стало бы фактором, существенно ограничивающим наши возможности. Но ведь можно обойтись и без специальной функции `interruptible_wait()`, так какими же другими вариантами можно воспользоваться? Одним из вариантов будет установка тайм-аута на ожидание, для чего вместо функции `wait()` можно воспользоваться функцией `wait_for()` с небольшим значением тайм-аута (например, 1 мс). Тем самым для потока будет задан верхний предел продолжительности ожидания, прежде чем он увидит наличие прерывания (с учетом промежутка между тактами часов). Если остановиться на этом варианте, ожидающий поток станет чаще ложно пробуждаться из-за тайм-аутов, но тут уж ничего не поделаешь. Реализация этого варианта, наряду с соответствующей реализацией `interrupt_flag`, показана в листинге 9.11.

Листинг 9.11. Использование тайм-аута в `interruptible_wait` для `std::condition_variable`

```
class interrupt_flag
{
    std::atomic<bool> flag;
    std::condition_variable* thread_cond;
    std::mutex set_clear_mutex;
public:
    interrupt_flag():
        thread_cond(0)
    {}
    void set()
    {
        flag.store(true, std::memory_order_relaxed);
        std::lock_guard<std::mutex> lk(set_clear_mutex);
        if(thread_cond)
```

```

        {
            thread_cond->notify_all();
        }
    }
    bool is_set() const
    {
        return flag.load(std::memory_order_relaxed);
    }
    void set_condition_variable(std::condition_variable& cv)
    {
        std::lock_guard<std::mutex> lk(set_clear_mutex);
        thread_cond=&cv;
    }
    void clear_condition_variable()
    {
        std::lock_guard<std::mutex> lk(set_clear_mutex);
        thread_cond=0;
    }
    struct clear_cv_on_destruct
    {
        ~clear_cv_on_destruct()
        {
            this_thread_interrupt_flag.clear_condition_variable();
        }
    };
};
void interruptible_wait(std::condition_variable& cv,
                      std::unique_lock<std::mutex>& lk)
{
    interruption_point();
    this_thread_interrupt_flag.set_condition_variable(cv);
    interrupt_flag::clear_cv_on_destruct guard;
    interruption_point();
    cv.wait_for(lk, std::chrono::milliseconds(1));
    interruption_point();
}

```

Если имеется некий ожидаемый предикат, то тайм-аут 1 мс может быть полностью скрыт внутри цикла предиката:

```

template<typename Predicate>
void interruptible_wait(std::condition_variable& cv,
                      std::unique_lock<std::mutex>& lk,
                      Predicate pred)
{
    interruption_point();
    this_thread_interrupt_flag.set_condition_variable(cv);
    interrupt_flag::clear_cv_on_destruct guard;
    while(!this_thread_interrupt_flag.is_set() && !pred())
    {
        cv.wait_for(lk, std::chrono::milliseconds(1));
    }
    interruption_point();
}

```

При этом предикат будет проверяться чаще обычного, но зато им легко будет воспользоваться вместо обычного вызова функции `wait()`. Песложно реализовать и другие варианты с использованием тайм-аутов: ждать, пока не истечет указанное время или 1 мс в зависимости от того, что короче. Итак, с ожиданием на `std::condition_variable` вопрос решен, а как обстоят дела с `std::condition_variable_any`? Там все точно так же или есть более удачные решения?

9.2.4. Прерывание ожидания на `std::condition_variable_any`

Класс `std::condition_variable_any` отличается от `std::condition_variable` возможностью работать с любым типом блокировки, а не только с `std::unique_lock<std::mutex>`. Ситуация упрощается, и с `std::condition_variable_any` можно найти более удачные решения, чем с `std::condition_variable`. Поскольку этот класс работает с любым типом блокировки, можно создать собственный тип, представляющий и снимающий как блокировку при работе с внутренним мьютексом `set_clear_mutex` в вашем `interrupt_flag`, так и блокировку, переданную при вызове функции `wait`, как показано в листинге 9.12.

Листинг 9.12. Реализация `interruptible_wait` для `std::condition_variable_any`

```
class interrupt_flag
{
    std::atomic<bool> flag;
    std::condition_variable* thread_cond;
    std::condition_variable_any* thread_cond_any;
    std::mutex set_clear_mutex;
public:
    interrupt_flag():
        thread_cond(0), thread_cond_any(0)
    {}
    void set()
    {
        flag.store(true, std::memory_order_relaxed);
        std::lock_guard<std::mutex> lk(set_clear_mutex);
        if(thread_cond)
        {
            thread_cond->notify_all();
        }
        else if(thread_cond_any)
        {
            thread_cond_any->notify_all();
        }
    }
}

template<typename Lockable>
void wait(std::condition_variable_any& cv, Lockable& lk)
{
    struct custom_lock
    {
        interrupt_flag* self;
```

```

    Lockable& lk;
    custom_lock(interrupt_flag* self_,
                std::condition_variable_any& cond,
                Lockable& lk_):
    self(self_),lk(lk_)
    {
        self->set_clear_mutex.lock();      ← ❶
        self->thread_cond_any=&cond;      ← ❷
    }
    void unlock()      ← ❸
    {
        lk.unlock();
        self->set_clear_mutex.unlock();
    }
    void lock()
    {
        std::lock(self->set_clear_mutex,lk); ← ❹
    }
    ~custom_lock()
    {
        self->thread_cond_any=0;      ← ❺
        self->set_clear_mutex.unlock();
    }
};
custom_lock cl(this,cv,lk);
interruption_point();
cv.wait(cl);
interruption_point();
}
// все остальное не изменилось
};
template<typename Lockable>
void interruptible_wait(std::condition_variable_any& cv, Lockable& lk)
{
    this_thread_interrupt_flag.wait(cv,lk);
}

```

Созданный только что тип блокировки получает блокировку на внутреннем мьютексе `set_clear_mutex` при его создании ❶, а затем устанавливает значение указателя `thread_cond_any` для ссылки на объект `std::condition_variable_any`, переданный в конструктор ❷. Ссылка на объект `Lockable`, доступ к которому уже должен быть заблокирован, сохраняется на будущее. Теперь можно проверить наличие сигнала на прерывания, не опасаясь, что возникнет состояние гонки.

Если в данном месте установлен флаг прерывания, значит, это было сделано до блокировки на мьютексе `set_clear_mutex`. Когда условной переменной внутри функции `wait()` вызывается ваша функция `unlock()`, снимается блокировка с объекта `Lockable` и с внутреннего мьютекса `set_clear_mutex` ❸. Это позволяет потокам, пытающимся прервать текущий поток, заблокировать мьютекс `set_clear_mutex` и проверить указатель `thread_cond_any`, как только управление окажется внутри вызова функции `wait()`, но никак не раньше. Это именно то, к чему мы стремились, но чего не смогли добиться при использовании `std::condition_variable`. Как

только ожидание в функции `wait()` завершится (либо из-за отсутствия уведомления, либо из-за ложного пробуждения), она вызовет функцию `lock()`, которая снова заблокирует внутренний мьютекс `set_clear_mutex` и объект `Lockable` ④. Теперь опять можно будет проверить наличие прерывания, появившегося в ходе вызова функции `wait()`, прежде чем удалить указатель `thread_cond_any` в деструкторе `custom_lock destructor` ⑤, где также снимается блокировка с мьютекса `set_clear_mutex`.

9.2.5. Прерывание других блокирующих вызовов

С прерыванием ожидания на условной переменной вопрос решен, а как насчет других ожиданий на блокировках: на заблокированных мьютексах, на готовности фьючерсов и т. д.? В принципе, можно обратиться к неиспользованному варианту с тайм-аутом, как уже было с `std::condition_variable`, поскольку нет простого способа прервать ожидание или выполнить ожидаемое условие без обращения к внутренним составляющим мьютекса или фьючерса. Но относительно этих ожиданий заранее известно, что именно ожидается, поэтому можно воспользоваться циклом внутри функции `interruptible_wait()`. В качестве примера рассмотрим переопределение функции `interruptible_wait()` для `std::future<>`:

```
template<typename T>
void interruptible_wait(std::future<T>& uf)
{
    while(!this_thread_interrupt_flag.is_set())
    {
        if(uf.wait_for(1k, std::chrono::milliseconds(1))==
            std::future_status::ready)
            break;
    }
    interruption_point();
}
```

Ожидание здесь продолжается до тех пор, пока не будет установлен флаг прерывания либо не будет готов фьючерс, но при каждом прохождении цикла блокирующее ожидание на готовности фьючерса продолжается 1 мс. Это означает, что в среднем сигнал на прерывание будет распознан примерно через 0,5 мс (для системных часов с довольно высокой частотой тактов). Ожидание в функции `wait_for` обычно длится по меньшей мере в течение одного такта системных часов, поэтому, если такт составляет 15 мс, ожидать придется не одну, а около 15 мс. Приемлемо это или нет, зависит от конкретных обстоятельств. При необходимости тайм-аут можно уменьшить (если системные часы позволяют), но тогда поток будет чаще пробуждаться для проверки установки флага, что увеличивает издержки на переключение задач.

Итак, мы рассмотрели возможности обнаружения прерывания с помощью функций `interruption_point()` и `interruptible_wait()`. А как обрабатываются прерывания?

9.2.6. Обработка прерываний

С позиции прерываемого потока прерывание представляется исключением `thread_interrupted`, которое в таком случае может быть обработано, как и любое другое исключение. В частности, его можно перехватить в стандартном блоке `catch`:

```
try
{
    do_something();
}
catch(thread_interrupted&)
{
    handle_interruption();
}
```

То есть исключение можно перехватить, неким образом обработать, после чего работу можно продолжить в обычном порядке. Если так и сделано и другой поток снова станет вызывать `interrupt()`, ваш поток опять прервется при следующем вызове точки прерывания. Такое развитие событий может потребоваться при выполнении серии независимых друг от друга задач: прерывание выполнения одной задачи вызовет ее отставку и поток сможет перейти к выполнению следующей задачи, имеющейся в списке.

Поскольку `thread_interrupted` является исключением, то при вызове кода, выполнение которого может быть прервано, во избежание утечки ресурсов и несогласованности в состоянии вашей структуры данных должны быть предприняты все обычные меры безопасности выдачи исключений. Зачастую желательно дать прерыванию завершить поток, позволив исключению распространиться вверх по цепочке вызовов. Но если разрешить исключениям распространяться за пределы функции потока, переданной конструктору `std::thread`, будет вызвана функция `std::terminate()` и прекращено выполнение всей программы. Чтобы не пужно было помнить о необходимости помещения обработчика `catch(thread_interrupted)` в каждую функцию, передаваемую `interruptible_thread`, можно вместо этого поместить блок `catch` внутрь оболочки, используемой для инициализации флага прерывания `interrupt_flag`. Тем самым будет обеспечена безопасность распространения исключения прерывания в необработанном виде, так как завершение коснется только данного потока. Теперь инициализация потока в конструкторе `interruptible_thread` будет такой:

```
internal_thread=std::thread([f,&p]{
    p.set_value(&this_thread_interrupt_flag);
    try
    {
        f();
    }
    catch(thread_interrupted const&)
    {}
});
```

А теперь рассмотрим конкретный пример, в котором может пригодиться прерывание.

9.2.7. Прерывание фоновых задач при выходе из приложения

Представьте себе приложение поиска на настольном компьютере. Кроме общения с пользователем, ему нужно отслеживать состояние файловой системы, обнаруживая любые изменения и обновляя свой индекс. Обычно, чтобы избежать отрицательного воздействия на отзывчивость графического пользовательского интерфейса, такая обработка возлагается на фоновый поток. Он должен быть запущен в течение всего жизненного цикла приложения, его запуск становится частью инициализации приложения, и он остается в рабочем состоянии до выхода из приложения. Выход из таких приложений обычно выполняется только при выключении машины, поскольку приложение должно работать все время, чтобы иметь возможность поддерживать актуальность индекса. В любом случае при завершении приложения нужно аккуратно закрыть все фоновые потоки. Один из способов — прерывание потоков.

В листинге 9.13 показана примерная реализация той части системы, которая управляет потоками.

Листинг 9.13. Отслеживание состояния файловой системы в фоновом режиме

```
std::mutex config_mutex;
std::vector<interruptible_thread> background_threads;
void background_thread(int disk_id)
{
    while(true)
    {
        interruption_point();
        fs_change fsc=get_fs_changes(disk_id);
        if(fsc.has_changes())
        {
            update_index(fsc);
        }
    }
}
void start_background_processing()
{
    background_threads.push_back(
        interruptible_thread(background_thread,disk_1));
    background_threads.push_back(
        interruptible_thread(background_thread,disk_2));
}
int main()
{
    start_background_processing();
    process_gui_until_exit();
    std::unique_lock<std::mutex> lk(config_mutex);
    for(unsigned i=0;i<background_threads.size();++i)
    {
        background_threads[i].interrupt();
    }
}
```

```

for(unsigned i=0;i<background_threads.size();++i)
{
    background_threads[i].join(); ← ❸
}

```

Фоновые потоки запускаются в самом начале ❶. Затем основной поток занимается обработкой графического интерфейса пользователя ❷. После того как пользователь запросит выход из приложения, фоновые потоки прерываются ❸, а основной поток, прежде чем выполнить выход, дожидается завершения каждого фонового потока ❹. В фоновых потоках запускаются циклы, в которых отслеживаются изменения на диске ❺ и выполняется обновление индекса ❻. При каждом проходе цикла путем запуска `interruption_point()` проверяется наличие прерывания ❼.

А зачем прерывать все потоки перед ожиданием завершения какого-либо из них? Почему бы не прерывать работу каждого из них, а затем дожидаться его завершения, перед тем как перейти к следующему потоку? Ответ кроется в *конкурентности*. Вероятнее всего, работа потоков не будет завершена сразу же, как только они получат сигнал на прерывание, поскольку им придется дойти до следующей точки прерывания, а перед выходом вызвать какой-либо деструктор и необходимый код обработки исключений. Немедленное объединение с каждым потоком заставит прерывающий поток входить в режим ожидания, даже если у него еще есть работа, которую он может выполнять (имеется в виду прерывание других потоков). В ожидании можно входить, лишь когда нечем будет заняться (все потоки уже прерваны). Это позволит всем прерываемым потокам обработать свои прерывания в параллельном режиме, что может ускорить завершение работы.

Такой механизм прерывания можно без особого труда расширить, добавив дополнительные прерываемые вызовы или отключив прерывания в определенном блоке кода, но это предлагаю вам выполнить самостоятельно.

Резюме

В этой главе мы рассмотрели продвинутые приемы управления потоками: использование пулов потоков и прерывание потоков. Было продемонстрировано, как использование локальных очередей работ и перехват работ может сократить издержки на обеспечение синхронизации и повысить пропускную способность пула потоков. Кроме того, мы разобрались, как выполнение других задач из очереди в ходе ожидания завершения подзадачи может исключить возможность взаимоблокировки.

Мы также разобрали различные способы, позволяющие одному потоку прервать работу другого, в частности применение конкретных точек прерывания и функций, которые допускают прерывание во время блокирующего ожидания.